FALCON: Relevance Feedback in Multimedia Databases[1]

Leejay Wu Christos Faloutsos Katia Sycara Terry R. Payne

June 13, 2001

[1]This research was partially funded by the National Science Foundation under Grants No. DMS-9873442, and IIS-9910606; also, by the National Science Foundation, ARPA and NASA under DARPA/ITO through Order F463, issued by ESC/ENS under contract N66001-97-C-851; and the Office of Naval Research Grant N-00014-96-1-1222. Additional funding was provided by donations from NEC and Intel.

Abstract

Several methods currently exist that can perform relatively simple queries driven by relevance feedback on large multimedia databases. However, all these methods work only for vector spaces; they require that objects be represented as vectors within feature spaces. Moreover, their implied query regions are typically convex. Our method, on the other hand, is the first method able to handle disjunctive queries within metric spaces, and it does so quickly and correctly. Given a pairwise distance metric, examples, and feedback, the system learns a composite distance function which models the implied concept. FALCON is independent of any feature space and the nature of the data or its distance function.

We also present fast, scalable algorithms for performing these searches. Taking advantage of existing metric indices such as M-trees, we derive rapid methods for both range and nearest-neighbor queries. Our prototype implementation shows promising performance. A 50-NN query with 5 query points scales well, requiring 5.17 seconds on 25,000 objects and 6.67 with 100,000. Moreover, it quickly converges, yielding high precision and recall, often within 5 to 10 iterations.

Keywords: Databases, Information Retrieval, Multimedia

1 Introduction

Querying by example is crucial in multimedia databases, where query concept complexity makes it difficult to fully specify a query with anything less than examples. This paper describes the FALCON system, designed from the ground up to be able to handle disjunctive queries.

Consider the following situation. A user stands before a terminal providing access to a stock tracking database. Suppose that he is only interested in stocks which have jumped up then down, or down then up. For reasons of simplicity, the terminal accepts input through examples drawn with a stylus and a touch screen, so the user draws the two images shown in Figure 1.



Figure 1: A hypothetical query.

We note that this example-driven query is disjunctive. Any system which averages the two query objects would be expected to hypothesize an ideal query point corresponding to a flat stock – which is exactly what the user does *not* want. We refer to any method which either requires exactly one query point, or accepts multiple examples but reduces them to one "ideal" query point as a single-query-point method.

In addition, *no* features have been specified by the user; just examples. Posit that the user, never having learned a query language, selects examples and marks them as "good" or "bad". Can we handle his query quickly and accurately?

Our system was designed to handle this siutation.

Problem Summary	
Given	A vector or metric data set.
	A non-empty set of examples labelled "good".
	A possibly empty set of examples labelled "bad".
Find	Other desirable objects from that data set.
Notes	The query concept may be disjunctive.
	Interactivity requires good performance.

FALCON, the Feedback Adaptive Loop for Content-Based Retrieval system, was designed for this domain. At its heart is a function called "aggregate dissimilarity", which uses the underlying distance metric to take into account all pairwise distances between a candidate and "good" examples. Its exact definition and properties will be described later on in this paper.

Practical applications abound. We believe that it would be useful for such disparate domains as video (Christel *et al.*, 1995); still images (Chiueh, 1994; Cox *et al.*, 1996; Cox *et al.*, 1998; Faloutsos *et al.*, 1994; Katayama & Satoh, 2000; Prokaew *et al.*, 1999; Rui *et al.*, 1998b; Rui *et al.*, 1998a); and time series (Jagadish *et al.*, 1999; Keogh & Pazzani, 1998; Keogh & Pazzani, 1999).

One example is a complicated search concept, such as video footage featuring heads of state, applied to a large multimedia database such as that used by the Informedia Project[1]. A pairwise distance metric between clips would be used by FALCON in order to find video clips similar to examples provided by the user, in such a way that it takes into account all such examples and thus avoids devolving to a single-query-point system. Even though the ideal isosurfaces that correspond to such a query may be very disjoint indeed, the aggregate dissimilarity function is flexible enough that sufficient feedback could indeed generate a reasonable approximation. All of this could be controlled with a simple interface making minimal demands upon a user.

As for evaluation, we have implemented a prototype system which was applied to objective

[1] `http://informedia.cs.cmu.edu`

queries on both real and synthetic data. For testing quality of results, we used the standard precision-recall metric; for testing speed, we considered both the number of positive examples provided and the size of the database.

The rest of this paper is organized as follows. Section 2 summarizes various related systems; the mechanisms underlying FALCON are presented in Section 3; the algorithms are in Section 4; the experiments and corresponding results are detailed in Section 5, and the subsequent analysis is presented in Section 6. The paper concludes with Section 7.

2 Related Work

A number of systems such as the following attempt to model queries using relevance feedback.

Rocchio's relevance feedback mechanism, and its variants, generate hyperspherical isosurfaces in feature space (Rocchio, 1971), (Buckley *et al.*, 1994). Hyperspheres by themselves do not generalize to disjunctive queries, and thus does not handle the more interesting queries.

PicHunter (Cox *et al.*, 1996), (Cox *et al.*, 1998), uses Bayesian methods to predict relevance and to take into account possible user error. Its Bayesian model makes it more difficult to directly compare capabilities.

Keogh and Pazzani have researched relevance feedback with regard to time series (Keogh & Pazzani, 1998), (Keogh & Pazzani, 1999). Their work incorporates a number of features specialized for the domain, notably in handling transformations such as shifting and scaling. It does not appear fundamentally incompatible with our system in terms of adaptability to multiple-object queries.

MARS (Multimedia Analysis and Retrieval System) handles queries on images and includes a query expansion model that can weight features from multiple objects – but appears to do so in such a way that it reduces to a single ideal query point (Rui *et al.*, 1998a; Prokaew *et al.*, 1999; Rui *et al.*, 1997; Rui *et al.*, 1998b).

MindReader uses the Mahalanobis distance to allow arbitrarily oriented ellipsoids (Ishikawa

et al., 1998). The Mahalanobis distance $M(\vec{x}, \vec{y})$ is defined as $(\vec{x} - \vec{y})^T \times \mathbf{M} \times (\vec{x} - \vec{y})$ where $\vec{x}$ and $\vec{y}$ are n-dimensional column vectors and $\mathbf{M}$ is a $n \times n$ covariance matrix. This provides a scaled and rotated Euclidean distance, but shares the disadvantage of being a single-query-point method; in addition, it applies only to vector spaces.

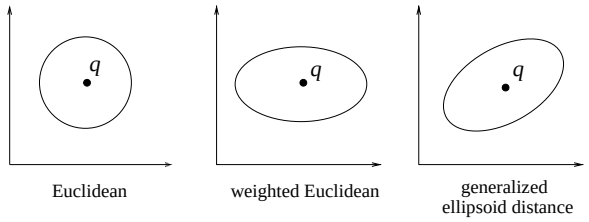


Figure 2: Three common types of isosurfaces. They are used by Rocchio methods, MARS, and MindReader respectively.

Figure 2 shows three common types of isosurfaces – spheres, orthogonally aligned ellipsoids, and arbitrarily aligned ellipsoids, where each isosurface is a set of points with equal "goodness" according to the query representation. The first permits translation; the second adds axis scaling; the third, rotation. However, these are all based on reducing to a single ideal query point, possibly augmented with transformed axes. Such methods fail on disjunctive queries, as shown by Figure 3..

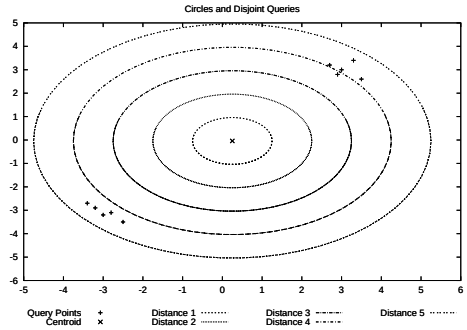


Figure 3: Single-query-point systems cannot handle disjoint queries.

Fox and Salton used the L_p metric to surpass fuzzy Boolean methods in the domain of text

retrieval, replacing the use of minimum and maximum functions. In that domain, disjunctiveness is important as one might desire documents that use any of a number of synonyms or closely-related terms, but do not require all (Salton *et al.*, 1983). Our objective is similar, but more complex; we wish to model arbitrarily disjunctive example-based queries in unbounded metric spaces using relevance feedback but lacking specific features (Salton *et al.*, 1983).

3 Proposed Method

This section describes the proposed method and underlying assumptions made by FALCON. Table 1 lists the notation used.

What FALCON does differs from previous methods in the following aspects. FALCON can handle metric spaces; works purely with examples and weights; and even models disjunctive queries.

As noted previously, we also provide algorithms that can take advantage of indexing structures such as R-trees (Guttman, 1984), R*-trees (Beckmann *et al.*, 1990; Brinkhoff *et al.*, 1993), M-trees (Ciaccia *et al.*, 1997), and Slim-trees (Traina Jr. *et al.*, 2000) that can perform fast, scalable searches in vector or metric space. In particular, we will make use of support for two common types of queries: range queries, which request all objects within a given distance of a single query object; and nearest-neighbor queries, which request the k objects closest to a given query point.

Let $\mathcal{X}$ be the set of objects in our metric data set. Then, let $d : \mathcal{X} \times \mathcal{X} \to \Re$ be the provided pairwise distance function.

To start each query, the user specifies at least one "desirable" example that is representative of the intended query. This set of "good" examples is denoted by $\mathcal{G}$. Thus:

Problem 1: Query by Multiple Examples	
Given	$\mathcal{X} = x_i$, a metric data set
	d, its pairwise distance metric
	$\mathcal{G} = \{g_i\}$, the set of "good objects"
Find	Other desirable objects in $\mathcal{X}$ that are similar to $\mathcal{G}$.

Symbol	Description
$\mathcal{X}$	The set of objects in our data set.
x	Any single object from $\mathcal{X}$.
$\mathcal{G}$	The current set of user-specified "good points".
g_i	A member of $\mathcal{G}$.
$D_{\mathcal{G}}$	The aggregate dissimilarity function based on $\mathcal{G}$; $D_{\mathcal{G}} : \mathcal{X} \longrightarrow \mathcal{R}^+ \cup \{0\}$
$D_{\mathcal{G}}(x)$	The aggregate dissimilarity value for object x to the current good set $\mathcal{G}$.
α	A constant that influences how $D_{\mathcal{G}}$ behaves.
d	The pairwise distance function used by $D_{\mathcal{G}}$.

Table 1: Notation used within this paper.

FALCON approaches this problem by creating a scoring function that designed to model the user's query; it generates a function $D_{\mathcal{G}} : \mathcal{X} \rightarrow \Re$ based on $\mathcal{G}$, such that this function varies inversely with the estimated desirability of x as judged by similarity to $\mathcal{G}$. $\mathcal{G}$, in turn, is modified as a consequence of user feedback.

3.1 Proposed "Aggregate Dissimilarity" Function

More formally, we have Problem 2. Given a pairwise distance metric, a set of good examples, and a candidate, how should its compute aggregate dissimilarity?

Problem 2: Aggregate Dissimilarity	
Given	$x \in \mathcal{X}$, a candidate
	$\mathcal{G} = \{g_i\}$, the set of user-selected "good objects"
	d, pairwise distance metric for $\mathcal{X}$
Determine	$D_{\mathcal{G}}(x)$, its aggregate dissimilarity

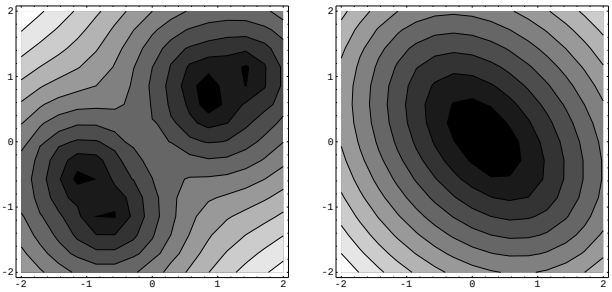
Once we have a function that fulfills our requirements – namely, ranking objects inversely according to their apparent desirability as compared to the set of good objects $\mathcal{G}$ – we have a space suitable for searching. If precision/recall evaluation is desired, we can compute this function on the entire sample and sort; if a user is waiting, we can perform a fast nearest-neighbor search and return however many objects were desired. If a range was implicitly

specified by a user specifying "marginal" objects, a range query could be performed as well. However, we first have to define this function.

We define the aggregate dissimilarity function with respect to $\mathcal{G}$, $D_{\mathcal{G}}(x)$, as follows.

$$(D_{\mathcal{G}}(x))^{\alpha} = \begin{cases} 0 & \text{if } (\alpha < 0) \bigwedge \exists i \; d(x, g_i) = 0 \\ \frac{1}{k} \times \sum_{i=1}^{k} d(x, g_i)^{\alpha} & \text{otherwise} \end{cases} \tag{1}$$

If the value of α is very high, the highest distance will have the largest impact on $D_{\mathcal{G}}(x)$, while the reverse is true for very low values of α. Results are undefined for $\alpha = 0$.



$(a)\alpha = -5$ $(b)\alpha = 5$

Figure 4: Single-query-point systems cannot handle disjoint queries.

Sample isosurfaces are shown in Figures 4 (a) and (b), where isosurfaces indicate continuous surfaces of equal $D_{\mathcal{G}}(x)$ value. Both (a) and (b) use the same axes and the same value of $\mathcal{G}$, differing only in α; (a) uses $\alpha = -5$, while (b) uses $\alpha = 5$.

Note that Equation 1 has the following properties:

- As the user modifies the set $\mathcal{G}$ of desirable objects, the function changes to reflect this.
- Negative values of α mimic a fuzzy OR function; positive values mimic a fuzzy AND function. See Appendix A for details and proof.
- For $\alpha \leq 0$, d such that $\forall x \; d(x, x) = 0$ and $x \in \mathcal{G}$, then $D_{\mathcal{G}}(x) = 0$ – in other words, with a fuzzy OR, exactly matching a single $x \in \mathcal{G}$ yields the best possible score – even if we

generalize $D_{\mathcal{G}}$ to accept weights, as described later. With $\alpha > 0$, FALCON's behavior is closer to that of MARS in that objects need to be similar to everything in $\mathcal{G}$ to minimize computed distance (Prokaew *et al.*, 1999).

- $\alpha = -1$ yields the harmonic mean of the pairwise distances. We note that the harmonic mean has been proposed for use in an alternative to the K-Means algorithm (Zhang, 2001).
- $\alpha = 1$ yields the arithmetic mean of the pairwise distances.
- $\alpha = 2$ produces hyperspheric isosurfaces around the centroid of the set. See Appendix B for details and proof.

Since it is not obvious which values of α, if any, provide optimal results, we empirically compare results for several different choices. From the point of supporting disjunctive queries, it is natural to suspect that a negative α would be better than a positive one, as we do not want to essentially require that candidates be near all "good" objects to achieve good scores. Our experiments show that, for all the different data and queries we used, $\alpha = -5$ is a reasonable choice.

3.2 An example

Recall our example of a hypothetical user who wishes to find stocks which, for a given time period, either went up then down, or down then up. The logical first step would be for the user to first choose a database containing stock price data. This data set should already have been assigned a distance metric.

Perhaps unsure about the capabilities of the system, the user then selects one example of the first pattern. Given a simple metric, this only returns part of the truly desired examples – namely, stocks that climb, then fall.

Upon realizing this, the user provides feedback by selecting an example of the second

pattern. The response then improves by incorporating both stocks that climb then fall, and stocks that fall then climb.

Figure 5 illustrates the relevance feedback loop. The initial choice of good examples forms the first $\mathcal{G}$. At each iteration, the current $\mathcal{G}$ is used to compute the aggregate dissimilarity function $D_{\mathcal{G}}$, which then is applied to the database to yield ranked examples. These examples are presented to the user, who provides feedback which can continue the cycle with an improved $\mathcal{G}$.

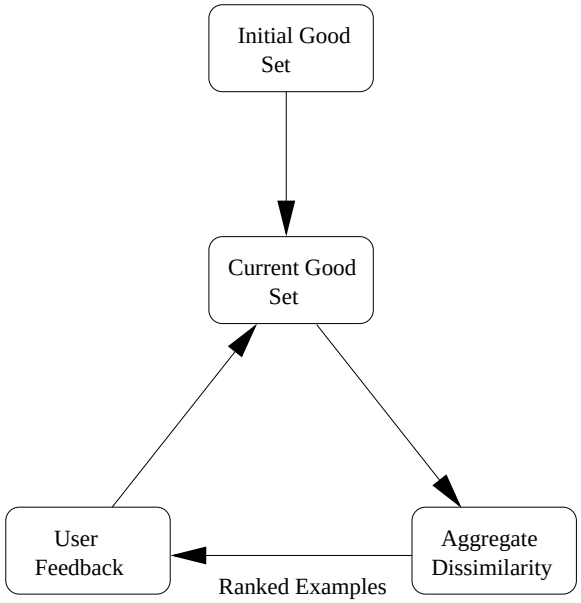


Figure 5: A high-level depiction of the FALCON process.

Figure 6 shows a prototype implementation working on a database of stock price time sequences. The top row of panels show the chosen examples; all four include a sharp rise followed by a fall, or its reverse. The bottom row of panels provides a scrollable display sorted according to rank; the instances shown were considered among the very best. The various buttons allow for choosing the "good" set as well weighting each member of that set. It must be emphasized that this query is disjunctive, and that other methods would face severe difficulties.

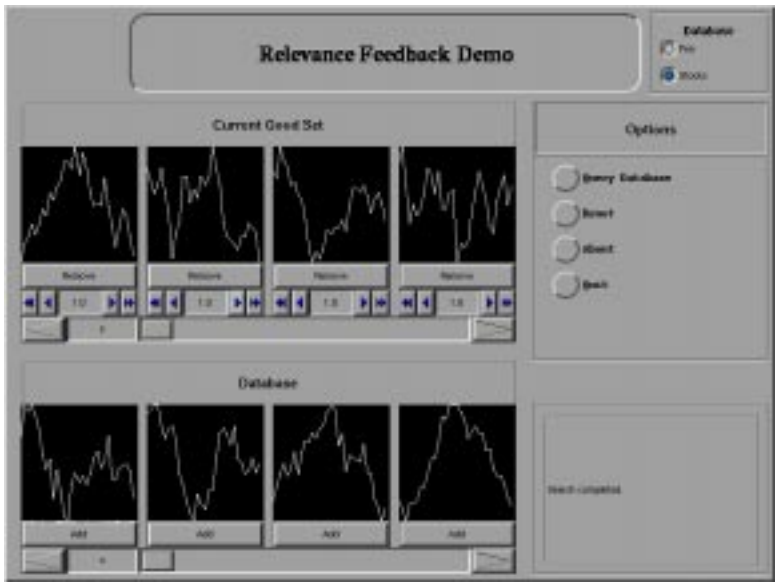


Figure 6: This shows our prototype working on the STOCKS data set.

3.3 Weighting examples

One useful generalization of the FALCON aggregate dissimilarity function is to allow weights. Unlike a distance "combination" function that uses only the minimum distance or maximum distance, the FALCON distance is easily modified to accept a w_i term for numerical feedback from a user. The proposed extension takes the form:

$$(D_{\mathcal{G}}(x))^{\alpha} = \frac{1}{\sum_{i=1}^{k} w_i} \cdot \sum_{i=1}^{k} w_i (d(x, g_i))^{\alpha} \tag{2}$$

For now, we require that $\forall i \; w_i > 0$, which avoids division by zero and other assorted difficulties. We note that with $\alpha = 1$, this function bears a strong structural resemblance to the similarity summation formula used in (Prokaew *et al.*, 1999).

This has the effect that distance to the favored objects is penalized less if α is negative – but not drastically so. We do *not* raise the weights to the αth power instead, since this would have the opposite effect, and thus would be much less intuitive. This allows us to retain the influence of all pairwise distances between candidate and examples in all cases except where

an exact match supercedes all other distances under a negative α.

4 Algorithms

Herein we describe our aforementioned algorithms for quickly, scalably performing both range and nearest-neighbor queries. As previously noted, the system is interactive and therefore rapid response is required. This suggests a need for sublinear query time if the system is to be applied to large databases.

4.1 Range queries

Problem 3: Range Query by Multiple Example	
Given	$\mathcal{X}$, the database
	$\mathcal{G} = \{g_i\}$, the set of "good objects"
	$\epsilon > 0$, a threshold
	d, pairwise distance metric for $\mathcal{X}$
Find	Q, such that $Q = \{x : x \in \mathcal{X} \wedge D_{\mathcal{G}}(x) < \epsilon\}$ quickly

There are numerous well-known indexing structures which support single-point range queries (Gaede & Günther, 1998). Given that support, we can rapidly perform their multi-query-point equivalents. Our algorithm would also work with an approximate range query algorithm, if that would yield superior speed. This may be of interest for locally metric data which may not, globally, obey the triangle inequality. Even when the distance function is not metric, we could still be able to avoid a full sequential scan.

It may be non-obvious why range queries are important to FALCON, given that the space transformed by $D_{\mathcal{G}}$ is completely hidden from the user. The first reason is the ability to quickly perform range queries will allow us to efficiently complete k-nearest-neighbor queries as well. The second reason is that with an example-driven interface, it need not be necessary at all for a user to explicitly specify a range, or even to be aware that what he is asking is a range

query. One could design an interface in which some examples are labelled "excellent", others are labelled "borderline", and the desired range is computed from the $D_{\mathcal{G}}$ to the "borderline" examples using the "excellent" set as $\mathcal{G}$.

The FALCON Range-Query Algorithm

1: For every object g_i in $\mathcal{G}$:

  1: Perform a single-point range-query for ϵ and g_i using the underlying index.

  2: Let the set of objects returned be $\mathcal{Q}_i$.

3: Compute $\mathcal{Q} = \bigcup_i \mathcal{Q}_i$. We will prove that this set is a superset of the desired answer, and hence we can simply filter false positives.

4: For each object x in $\mathcal{Q}$:

  1: Compute $D_{\mathcal{G}}(x)$.

  2: If $D_{\mathcal{G}}(x) > \epsilon$, then remove x from $\mathcal{Q}$.

3: Return $\mathcal{Q}$.

Figure 7: The FALCON Range-Query Algorithm.

Figure 7 shows the algorithm for performing a range ϵ query with $\mathcal{G}$ as the set of query points. To prove its correctness, we first need to show that it results in no false dismissals.

Theorem 1 *Consider Problem 3 above. Executing $k = |\mathcal{G}|$ separate range queries, each with the same threshold, $\epsilon > 0$, will yield k sets, the union of which forms a superset of the actual answer. No matching object will be falsely discarded as a candidate by such a procedure.*

Proof: We shall proceed using proof by contradiction. Assume an object x has been falsely dismissed. That is, $D_{\mathcal{G}}(x) \leq \epsilon$, and yet it is not returned. Then, by definition of the returned set as the union of the sets resulting from each individual range query, we know that $\forall i \; d(x, g_i) > \epsilon$.

As we have excluded $\alpha = 0$ from consideration, there remain two cases; negative α and positive α.

Case 1: $\alpha < 0$**:** Then,

$$\forall i \; d(x, g_i)^{\alpha} < \epsilon^{\alpha} \tag{3}$$

Therefore,

$$\sum_i d(x, g_i)^{\alpha} < \sum_i \epsilon^{\alpha} \tag{4}$$

or, using our previously noted extension for weights,

$$\sum_i w_i d(x, g_i)^{\alpha} < \sum_i w_i \epsilon^{\alpha} \tag{5}$$

From this point, we continue only with the latter branch as the former is a special case of the latter with $\forall i \; w_i = 1$.

Divide both sides by $\sum_i w_i$, and we obtain

$$\frac{\sum_i w_i d(x, g_i)^{\alpha}}{\sum_i w_i} < \epsilon^{\alpha} \tag{6}$$

Then, computing the α-th root of both sides of the inequality yields

$$\left(\frac{\sum_i w_i d(x, g_i)^{\alpha}}{\sum_i w_i} \right)^{\frac{1}{\alpha}} > \epsilon \tag{7}$$

The left side is the exact definition of the weighted variation of $D_{\mathcal{G}}(x)$, which means that object x actually has an aggregate dissimilarity strictly greater than the threshold ϵ. Thus, we have a contradiction. For $\alpha < 0$, there can be no false dismissals.

Case 2: $\alpha > 0$**:** The proof of the latter case is isomorphic to the proof of the former, and requires only changing the direction of the inequalities in Equations 3 through 6.

Thus, the algorithm yields no false dismissals. **QED**. Since the last steps in the aggregate range query algorithm eliminate false positives by checking actual $D_{\mathcal{G}}$ values, the algorithm returns precisely the correct results.

4.1.1 k-Nearest-Neighbor Queries

Nearest-neighbor queries are also very popular. A user is often interested in simply getting the top five, ten or twenty closest matches. If we have an underlying index that supports both range and nearest-neighbor queries, we can incorporate the previous algorithm to satisfy this query as well.

Assume the same definitions, except that we have k, the number of neighbors to return; instead of ϵ, the range-query threshold. Furthermore, we require that the data structure used supports normal k-nearest-neighbor queries – that is, such queries involving but a single query point. Then, the algorithm shown in Figure 8 performs this task.

Theorem: The FALCON k-nearest-neighbor algorithm is guaranteed to be correct, given correct implementations of single-point range and nearest-neighbor queries.

The same lemma regarding range queries and the lack of false dismissals allows us to prove correctness of this algorithm as well.

Proof: For simplicity, assume that ties are broken using a deterministic but arbitrary algorithm, so that the algorithms need only return exactly k neighbors. The FALCON k-nearest-neighbor algorithm can trivially be extended to return more if there are multiple objects with identical $D_{\mathcal{G}}$ that all qualify to be the kth nearest neighbor.

There are two cases that we wish to show are impossible; the first is the erroneous absence of one of the k nearest neighbors, while the second is the erroneous inclusion of a object that is not one of the desired neighbors.

The FALCON k-Nearest-Neighbor Algorithm

1: For every object g_i in $\mathcal{G}$:

   1: Perform single-point k-NN for k and g_i.

   2: Let the set of objects returned be $\bar{\mathcal{Q}}_i$.

3: Compute $\bar{\mathcal{Q}} = \bigcup_i \bar{\mathcal{Q}}_i$. This set contains all the objects that are at worst a kth-nearest neighbor to at least one member of $\mathcal{G}$. We will use a heap to find the kth lowest $D_{\mathcal{G}}$ among members of $\bar{\mathcal{Q}}$.

4: Initialize a min-heap priority queue $\mathcal{H}$ of size k.

5: For each object x in $\bar{\mathcal{Q}}$, insert $D_{\mathcal{G}}(x)$ into $\mathcal{H}$.

6: Find the k-th smallest value in $\mathcal{H}$. This value $\bar{\epsilon}$ is an estimate for the $D_{\mathcal{G}}$ value to the k-th nearest neighbor.

7: Use the FALCON Range-Query Algorithm with $\mathcal{G}$ and $\bar{\epsilon}$, yielding $\mathcal{Q}$. Note that we had already found at least k members of $\bar{\mathcal{Q}} \in \mathcal{X}$ that qualify.

8: Empty heap $\mathcal{H}$.

9: For each object x in $\mathcal{Q}$, insert $D_{\mathcal{G}}(x)$ into $\mathcal{H}$.

10: Remove from $\mathcal{Q}$ all but the k objects with lowest $D_{\mathcal{G}}$ values, breaking ties arbitrarily.

11: Return $\mathcal{Q}$.

Figure 8: The FALCON k-Nearest-Neighbor Algorithm.

The two cases are actually equivalent when restricted to precisely k neighbors returned; for every "correct" neighbor that is not returned, there must have been an "incorrect" neighbor that was, and vice versa. Therefore, we need only to prove that one of the cases is impossible. For the purposes of proof by contradiction, assume the former case – that of the missing neighbor.

Let there be an object x that is one of the k nearest neighbors, but is not returned. It must have been rejected in Step 6, or accepted in Step 6 but pruned in Step 9.

If it did not pass Step 6, then it was rejected by the range query. As we have already proved correctness of the FALCON Range-Query Algorithm, we know that $D_{\mathcal{G}}(x) > \bar{\epsilon}$. However, $\bar{\epsilon}$ was defined in Step 5 in such a way that we had already identified k objects of lesser or equal $D_{\mathcal{G}}$, and these are therefore nearer neighbors than x. Given our assumption about breaking ties, x cannot be one of the k nearest neighbors, which is a contradiction.

The other qualifying step it must pass is Step 9. The only way to be pruned from $\mathcal{Q}$ once accepted by the range queries in Step 6 is for there to be k other objects with better $D_{\mathcal{G}}$, which again implies that x really is not one of the k nearest neighbors, and therefore results in a contradiction.

Therefore, an object x must be returned if it is indeed one of the k nearest neighbors, and therefore the algorithm returns exactly those neighbors. *QED.*

Therefore, FALCON can take full advantage of indexing structures, *without* needing to inspect or alter any of their internals. This can even work well for high-dimensional data, as many real data sets are quite skewed (Pagel *et al.*, 2000) and thus do not suffer from the theoretical issues described in (Beyer *et al.*, 1998).

Later in this paper we present empirical performance evaluation of these algorithms.

5 Experimental Setup

We implemented the core parts of FALCON in C, C++ and Perl, and tested using an Intel Pentium II^{TM} workstation running Linux. As noted before, Figure 6 shows a screenshot of one implementation in action.

We tested FALCON with the intent of answering five central questions.

(a) Does FALCON "learn" to model concave and disjunctive queries?

(b) Does FALCON provide satisfactory precision/recall?

(c) Does FALCON rapidly converge to a satisfactory level of precision/recall?

(d) What is a suitable value of α?

(e) How fast is FALCON, in terms of time required?

5.1 Data

Five data sets were used during the experiments. Three were used only for assessing the quality of results. The largest of the five, COVER, was used for testing our algorithms for taking advantage of existing support for single-query-point range and nearest-neighbor queries; and one data set, $2D_20K$, was used for both purposes.

Two of the data sets were synthetic; the other three consisted of real data. We used the standard Euclidean distance as the pairwise distance metric for each set; for the $2D_20K$ data set, we also experimented with L_∞.

2D_50K: This synthetic data set consists of 50,000 points in 2-dimensional Cartesian space, pseudorandomly distributed approximately uniformly within the axis-aligned square (-2,-2) - (2,2).

2D_20K: This synthetic data set was generated using identical rules to that of the 2D_50K data set, but consists only of 20,000 points.

PEN: This database was obtained from the UCI repository (Murphy & Aha, 1994), and consists of objects that correspond to handwritten digits. Each object is a 16-dimensional

vector consisting of eight pairs. Each pair contains the scaled Cartesian coordinates of a point sampled in time. All objects are labelled with the class, which is the corresponding digit.

STOCKS: Daily closing prices for periods of up to five years were collected for 51 stocks from Yahoo's online quote server. They were split into 1856 non-overlapping vectors of length 32, which were then processed via the discrete wavelet transform (DWT). All 32 coefficients for each vector were used for L_2 distance computations.

COVER: This forest cover database was obtained from the UCI repository (Murphy & Aha, 1994), and consists of 581,012 54-dimensional vectors.

5.2 Queries

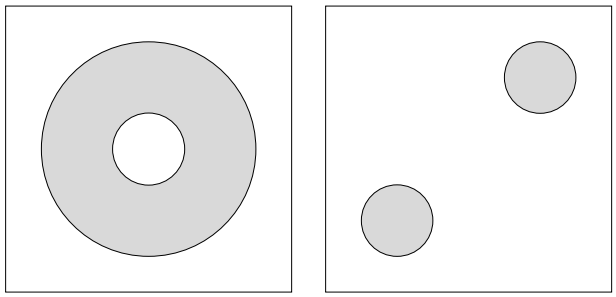
For quality testing, each data set was paired with a corresponding implicit user query. All such queries were objective, so that for evaluation of returned results one could check whether an example should or should not have been returned. For speed testing, we simply need a set of "good" objects to define the initial query, as feedback is not a direct concern.

The following describes the queries used, and provides information regarding their selectivity in the test and training data.

RING: We paired this query with the 2D_50K data set. Points were marked as positive examples if and only if they were between 0.5 and 1.5 units from the origin, inclusive, as shown in Figure 9(a). The five seeds were generated 1.4 units from the origin at integral multiples of 72 degrees counter-clockwise from the positive X axis. This query is unremarkable except for the undesired region within the middle. 19,734 points fulfilled our query.

TWO_CIRCLES: Points from 2D_20K were considered positive if and only if they were within 0.5 units of either (-1,-1) or (1,1), as shown in Figure 9(b). The seeds were randomly generated from within the two circles. This query was designed to test FALCON's ability to handle disjunctive queries. Clearly, any system designed to assume contiguous isosurfaces will have difficulty with this query. 1,899 points were positive examples.

PEN_4: For the PEN data set, we label positive those examples that had been classified



(a) RING (b) TWO_CIRCLES

Figure 9: The shape of the RING and TWO_CIRCLES queries are shown above. Bounding boxes denote range of data, and shaded areas indicate where examples would be labelled positive.

as 4's. The five seeds were drawn randomly from the positive instances in the training set. The intent here is to show that FALCON can answer a reasonable query on real data. 780 in the supplied "test" set were positive instances.

FLAT_STOCKS: The query over the STOCKS data set looks for approximately "flat" stocks. Hence, we classified examples as positive if and only if the slopes of their least-squares linear approximations were within the range [-0.02, 0.02]. The five seeds were the vectors of discrete wavelet transform (DWT) coefficients of five perfectly flat lines with y-intercepts of 8, 40, 80, 200 and 400. There were 540 positive examples in the full set.

COVER: The COVER set was used purely for scalability testing; consequently, we did not pick a specific concept for the query. Instead, five vectors were randomly chosen from the full set.

5.3 Evaluation

We now outline the methodology we used for evaluating FALCON.

5.3.1 Quality of Results

Figure 10 shows the relevance-feedback cycle used for quality testing. It differs from that of Figure 5 in the following aspects.

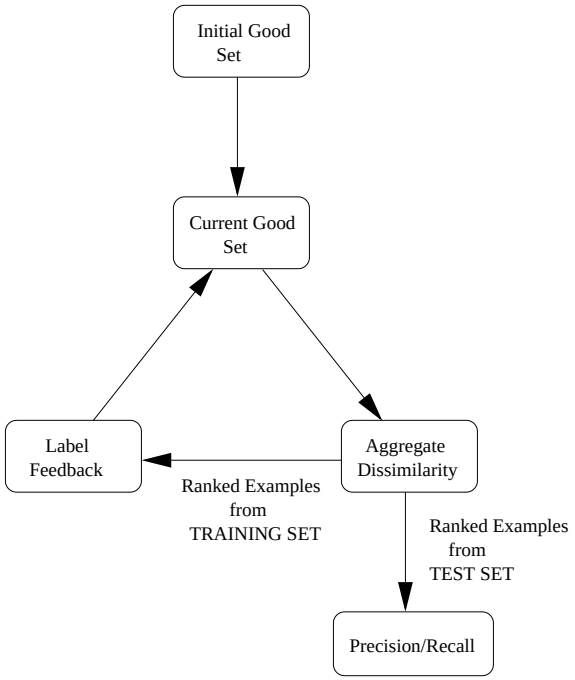


Figure 10: A high-level depiction of the FALCON evaluation process.

1. Since all the queries are objective, pre-computed labels are used instead of human-generated feedback.

2. The ranked examples presented for labelling come from a training set, which is a fixed but randomly-chosen subset of the data. The top twenty such training set members that had never been selected for feedback are used each iteration.

3. The same $\mathcal{G}$ being learned from the training set results in the same $D_{\mathcal{G}}$ which is applied to rank the test set – namely, the entire data set. This ranking is then used to calculate precision-recall.

Thus, there are now three sets per query; the seeds, the training set, and the test set. This split enables us to test whether our $D_{\mathcal{G}}$ is hurt by overfitting, as well as its ability to generalize when trained only on a sample.

These sets were allocated as follows.

RING: The training subset consisted of 1000 randomly selected points. 431 points in the subset fulfilled our query.

TWO_CIRCLES: The subset consisted of 1000 points randomly chosen from the full set, including 99 positive examples.

PEN_4: We used the existing split of 3498 objects in the training set and 7494 in the test set. 364 vectors in the smaller set qualified.

FLAT_STOCKS: The training subset consisted of 500 randomly drawn examples, including 153 positive examples.

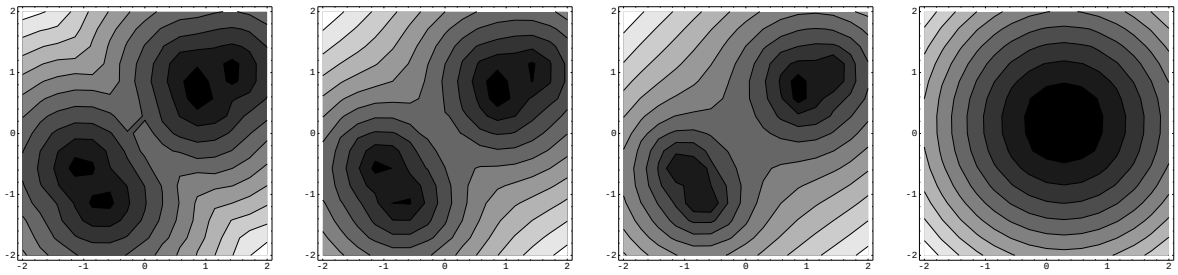
As noted above, the metric used is precision-recall. Consider the ranking generated on the test set. If among the r top-ranked objects are r_p positive examples, and there are n_p positive examples in the entire data set, then the precision is $\frac{r_p}{r}$ and the recall is $\frac{r_p}{n_p}$. For example, if within the top ten responses were seven positive examples, the precision is 70%; and if there were 20 positive examples in the entire set, the corresponding level of recall is 35%.

Precision-recall then is defined for a given level of recall as the precision given the *least* such r to achieve that particular level of recall. In the previous example, if the seven positive examples were also the seven top-ranked objects, the precision at 35% level of recall would be 100%. We measured precision-recall at 10% intervals.

5.3.2 Sensitivity of α

As noted previously, FALCON uses one parameter α. There is little theoretical reason to believe that one value of α would be optimal for all queries on all data sets, and we therefore tried multiple values.

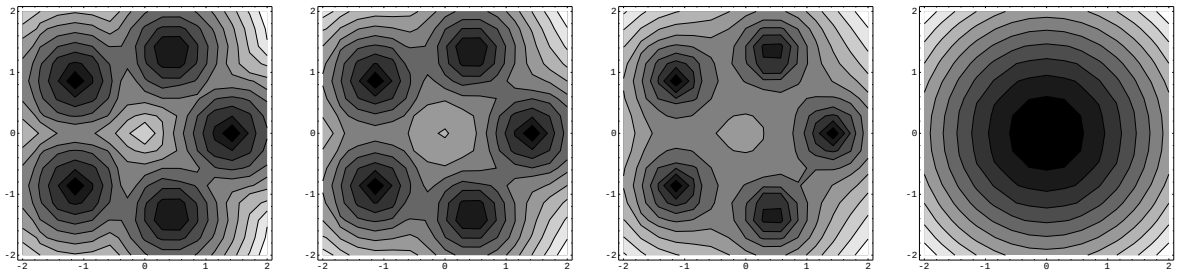
Figure 11 shows contour plots for four values of α using the TWO_CIRCLES seeds as $\mathcal{G}$.



$\alpha = -\infty$ $\alpha = -5$ $\alpha = -2$ $\alpha = +2$

Figure 11: Contour plots for the seeds of the TWO_CIRCLES query.

Note that $\alpha = +2$ reflects only the center of the "good set", even on the TWO_CIRCLES query, and hence is expected to fail on such a disjunctive query.



$\alpha = -\infty$ $\alpha = -5$ $\alpha = -2$ $\alpha = +2$

Figure 12: Contour plots for the seeds of the RING query.

Figure 12 shows contour plots for four values of α in the RING. Again, $\alpha = +2$ stands out with contours that are not going to rank the points very well. Thus, α matters to a certain degree; our tests assess this.

5.3.3 Speed

Elsewhere in this paper we presented algorithms for efficiently performing range and k-NN queries in aggregate dissimilarity space. Herein we describe our methods and their results.

We stored database keys using Patella's implementation of an M-tree, which we modified to

use the historic DB format supported by the Berkeley DBM library (Ciaccia *et al.*, 1997). The actual data lay within the DB files, while the M-tree nodes stored only the keys. Thus, this was a non-clustering indexing scheme where the DB file did not take advantage of organization by the M-tree. We speculate that organizing data storage accordingly would improve performance.

Our results for range queries may differ very slightly from those quoted in the original technical report due to source-level tweaks and compiler upgrades (Wu *et al.*, 2000). We note, however, that the general trends remain the same.

In all cases, α was fixed to be -5.

Range queries: The query and data involved were TWO_CIRCLES and the training subset of 2D_20K used previously, with corresponding seeds. Sequential scanning served as a baseline with which to compare the method described in Theorem 1.

First, we varied the threshold ϵ while measuring both elapsed time and distance calculations. Second, we varied the number of seeds while keeping the threshold constant. Additional seeds were selected randomly from the positive examples.

k-NN queries: We performed similar tests for the k-NN algorithm, varying both database size and k. The databases were all independent random samples of the COVER database.

6 Results and Discussion

We present our analysis of the results, organized with regard to the five central questions listed in Section 5.

6.1 Notes on the graphs

For all the figures marked "Precision versus Recall", such as Figures 13, 15, and 20, one line is plotted per charted iteration; not all iterations are shown for readibility reasons. Each line connects ten points, each of which shows precision at one level of recall from 10% to 100% at 10% increments. Hence, the differences from line to line track the progress of FALCON as

feedback is provided on increasing numbers of points.

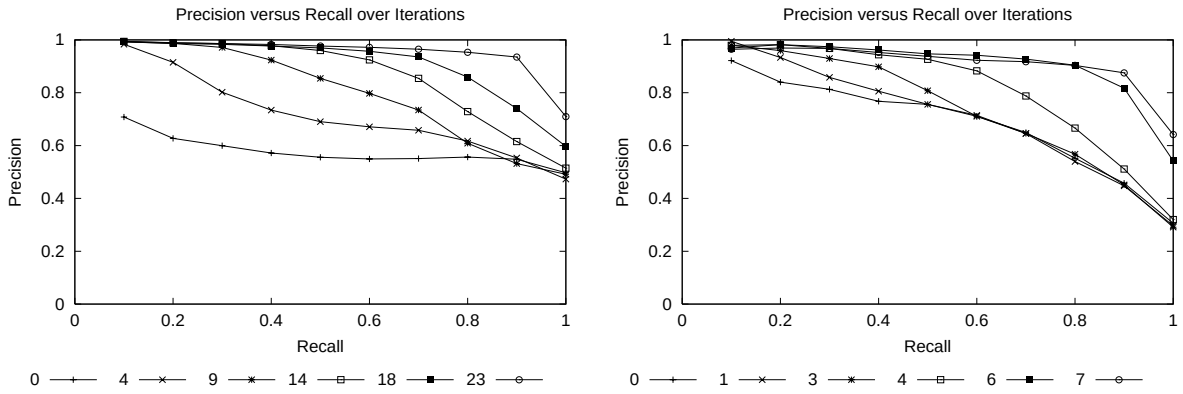
Any figure that tracks "Precision versus Iterations", such as Figures 16 and 17, instead focuses on precision at one level of recall, 40%, for multiple values of α. Positive slopes indicate positive progress.

Figures 21 and 22, labelled "Precision at Multiple Levels of α" also track precision, but after 5 and 20 iterations. The first shows precision at 40% recall; the second, average precision based on all 10 levels of recall.

In the graphs, precision and recall are always expressed as ratios from 0 to 1 with the aforementioned semantics.

6.2 Concave and Disjunctive Queries

The RING query is concave, but contiguous; the TWO_CIRCLES query is disjunctive. As Figure 13 shows, FALCON can handle both with high levels of precision versus recall. This is a substantial improvement over existing methods such as MARS and MindReader.



RING TWO_CIRCLES

Figure 13: Precision versus recall for $\alpha = -5$, for RING and TWO_CIRCLES, using Euclidean pairwise distance. As with similar graphs, the line numbers specify iterations.

The legend indicates which line corresponds to which iteration. In both cases, precision

stabilizes after the first several iterations – especially for TWO_CIRCLES.

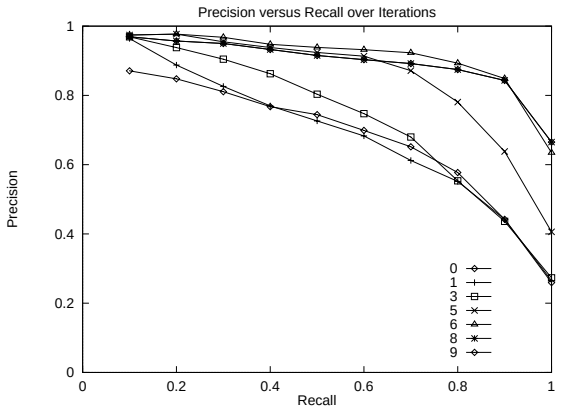


Figure 14: Precision versus recall for $\alpha = -5$, TWO_CIRCLES, and L_∞ as distance metric d

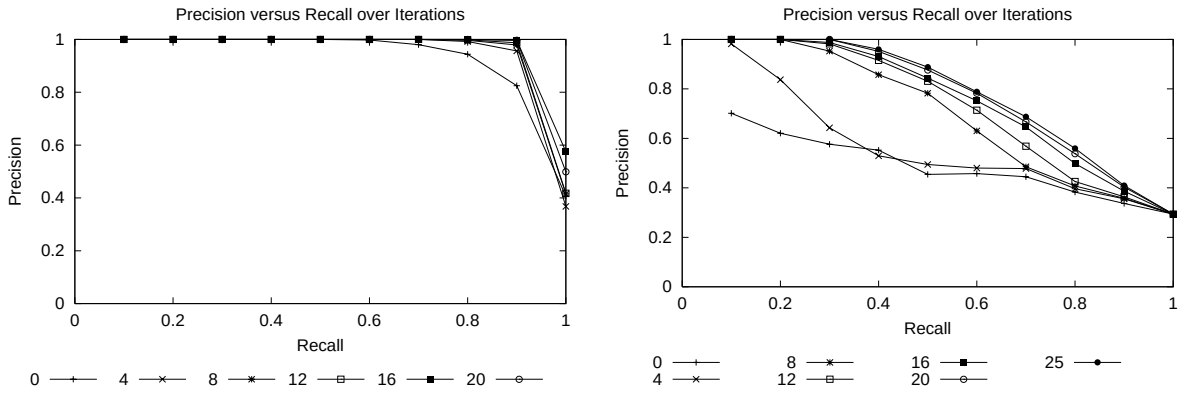
Success on the TWO_CIRCLES query is particularly notable because the set satisfying the query is disjunctive; there are two distinct regions of points that deserve high rankings, separated by points that do not. To further test the system, we ran the same query substituting L_∞ for the Euclidean distance as pairwise metric d. Figure 14 shows the resulting precision-recall.

6.3 Quality of Results on Real Data

Both the PEN_4 and FLAT_STOCKS queries are reasonable queries on real data. Consequently, FALCON's performance on these, as shown in Figure 15, is relevant to any question as to whether FALCON "works" on real data. The answer is clearly yes.

On the PEN_4 query, FALCON rapidly converges to very high precision at all levels of recall below 100% In fact, feedback iterations after the 4th do not add precision except at the highest level of recall. The pattern with the FLAT_STOCKS query is more interesting, with large gaps between the lines. One plausible explanation is that the positive examples were not evenly distributed in vector space, but instead clustered. The first member of a cluster to be added to $\mathcal{G}$ causes other members to move upwards in the rankings – at least when $\alpha < 0$ and

$D_{\mathcal{G}}$ resembles a fuzzy OR. This seems likely given that many of the stock vectors came from the same stocks, and some may have been affected by the same trends.



PEN STOCKS

Figure 15: Precision versus recall for $\alpha = -5$, for PEN_4 and FLAT_STOCKS.

6.4 Speed of Convergence

Referring to Figures 16 through 19, which show precision at 40% recall versus the number of iterations, we again see that FALCON quickly reaches good levels of precision on each query. FALCON can attain high levels of precision over high levels of recall, even with an early $\mathcal{G}$ that has not been substantially expanded.

The FLAT_STOCKS query is unlike the other queries

For PEN_4 and TWO_CIRCLES, the results for different values of α are quite similar to each other. Differences are more significant for RING and STOCKS, but even with these two sets, they appear to converge after a small number of iterations.

Depending upon the complexity of the query and the data set, progress can be very fast, as with the PEN_4 query, or somewhat slower, as with the RING query. For 40% recall, precision exceeds 90% after only 4 to 11 iterations for $\alpha = -5$ on all four queries.

There can be overfitting, as seen in the TWO_CIRCLES query; multiple iterations grow G

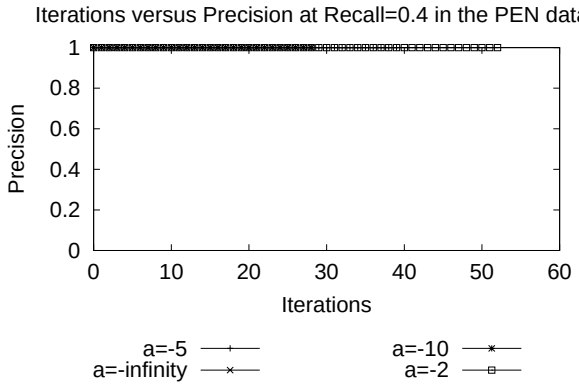


Figure 16: Precision versus iterations at recall=40% for PEN_4.

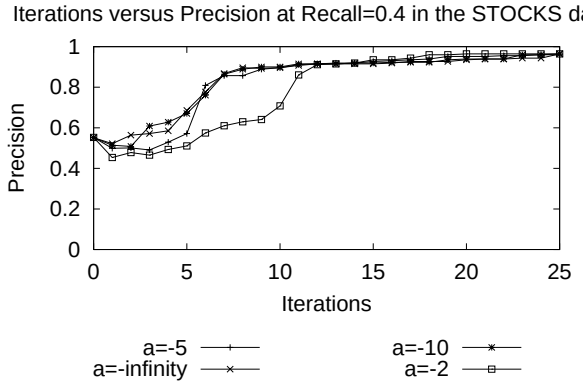


Figure 17: Precision versus iterations at recall=40% for FLAT_STOCKS.

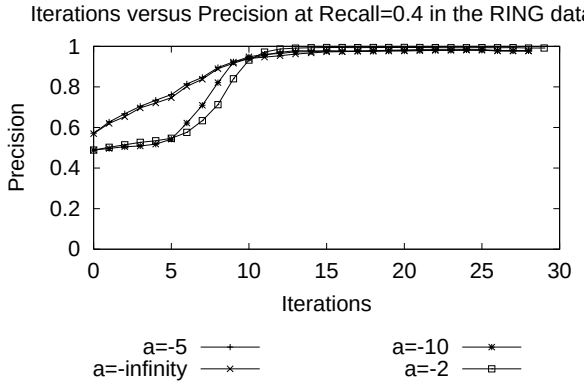


Figure 18: Precision versus iterations at recall=40% for RING.

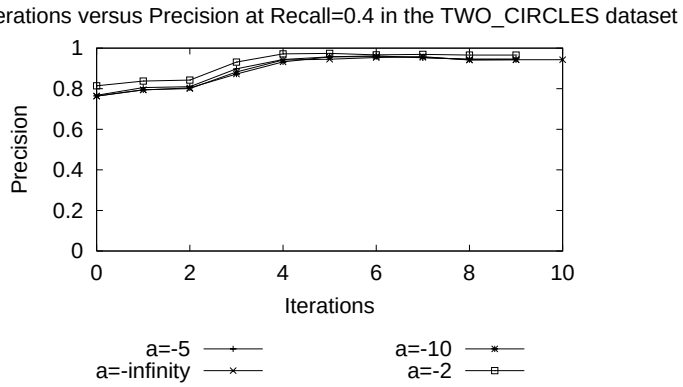


Figure 19: Precision versus iterations at recall=40% for TWO_CIRCLES.

without improving precision. The law of diminishing returns clearly applies to increasing $|\mathcal{G}|$.

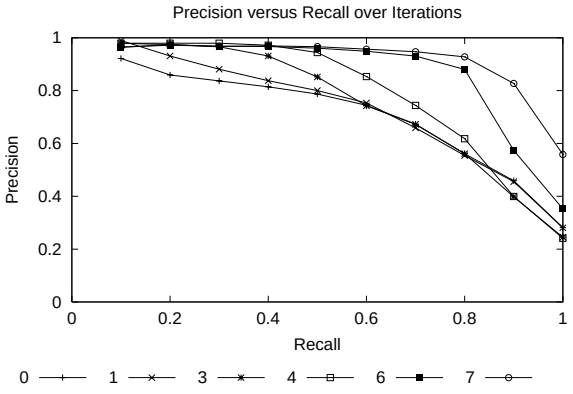
6.5 Optimal Value of α

We had theorized that $\alpha = -5$ would be a reasonable compromise balancing the desire to support highly disjunctive queries with the need to avoid overfitting. No single value was optimal for all queries, but our results show that $\alpha = -5$ is indeed reasonable, as are a range of negative values. Positive values are less useful, which is to be expected when considering disjunctive queries.

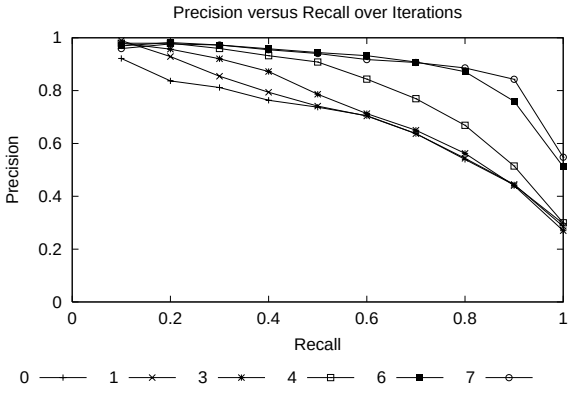
In our experiments, $\alpha = -100$ usually performed somewhat worse than $\alpha = -5$. A possible cause the use of finite-precision arithmetic; FALCON, with $\alpha = -100$, deals with unusually small numbers.

Figure 20 shows precision versus recall for the TWO_CIRCLES query for $\alpha = -2$ and $\alpha = -10$. As usual, the numbers in the legend count iterations. For both values, all positive examples points in the sample were added to $\mathcal{G}$ in no more than 9 iterations of feedback. We also note that the progress in precision over multiple levels of recall was fairly steady throughout.

Precision was generally similar for all negative values of α, especially once $\mathcal{G}$ had reached

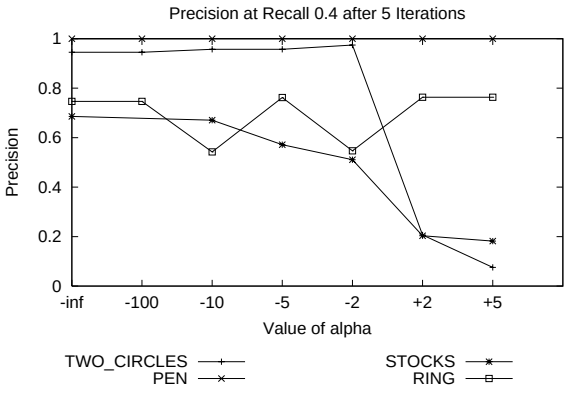


$(\alpha = -2)$

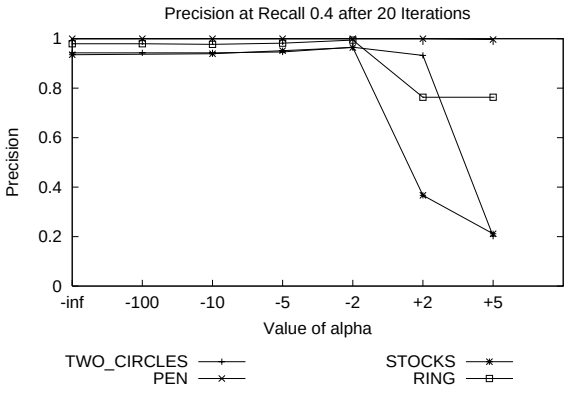


$(\alpha = -10)$

Figure 20: Precision versus recall on the TWO_CIRCLES query.

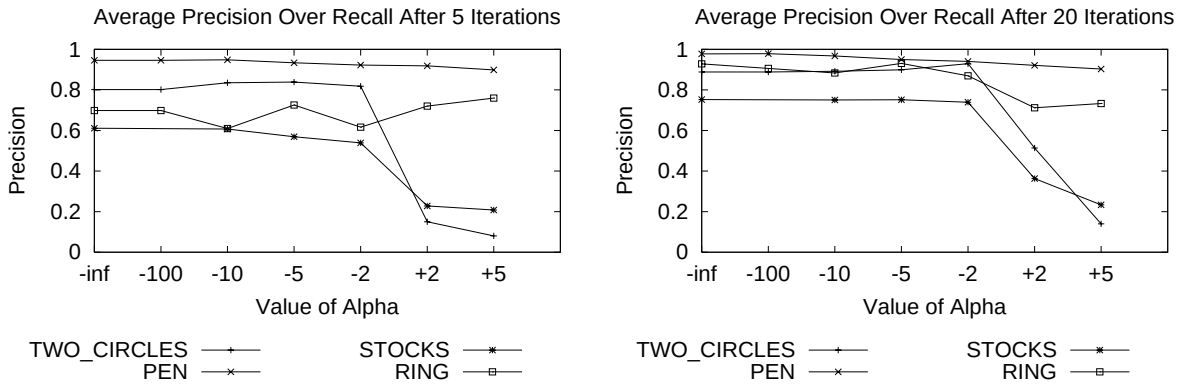


After 5 Iterations



After 20 Iterations

Figure 21: Precision at 40% recall



After 5 Iterations

After 20 Iterations

Figure 22: Average precision at multiple values of α.

its maximum size. With positive values of α, precision was quite low at most levels of recall; see Figures 21 and 22 for some results.

The first figure shows precision at 40% recall for each level of tested α; each line tracks the precisions yielded by one particular value of α for the different queries. This allows us to compare α both early and late in the process. In the case of TWO_CIRCLES, for which $\mathcal{G}$ stabilized in fewer than 20 iterations, we used the final results. Differences among the negative values of α largely disappear by the 20th iteration of feedback, suggesting that the exact value is not crucial.

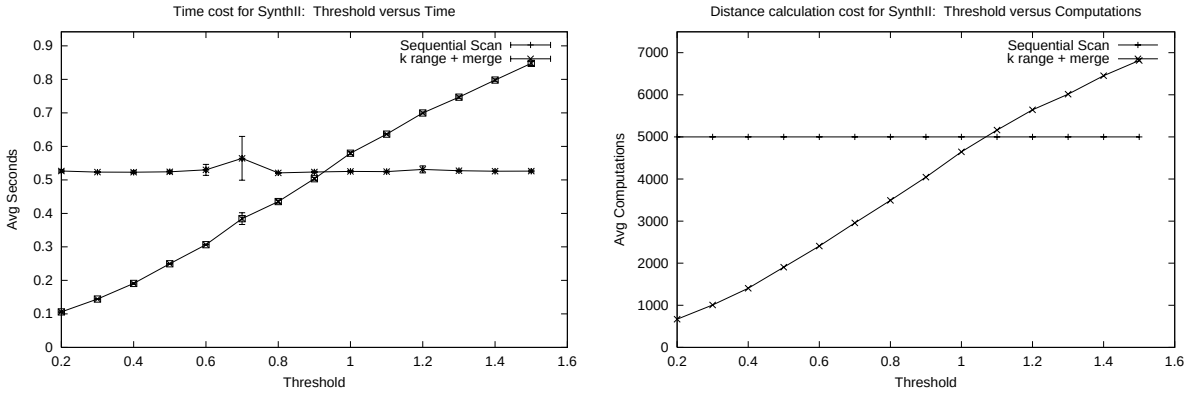
The latter figure shows that differences in average precision over all levels of recall after 20 iterations are a bit more marked than at 40%, but again similar values of α yield similar results. We therefore conclude that $\alpha = -5$ is a reasonable value, and that α is not a "magic" parameter; its sign matters, but otherwise its choice need not be of great concern.

6.6 Speed

Herein we describe the results of our speed tests – those involving range and k-NN queries.

6.6.1 Range queries

Empirical testing showed that merging the results of k separate range queries as per Theorem 1 scales very well in terms of both the number of distance computations and the elapsed time required. Some of these results may be noted in Figures 23 and 24.



Elapsed time

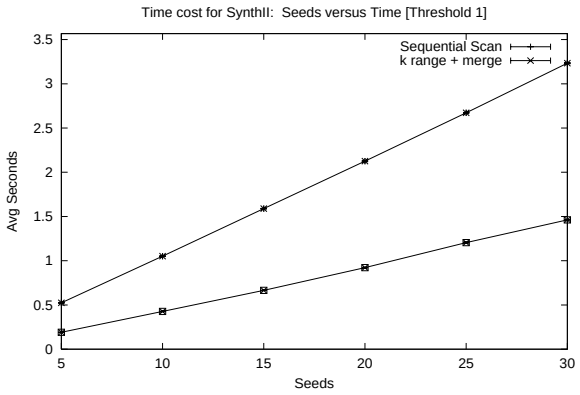
Distance Computations

Figure 23: Time and computational cost for TWO_CIRCLES, varying threshold.

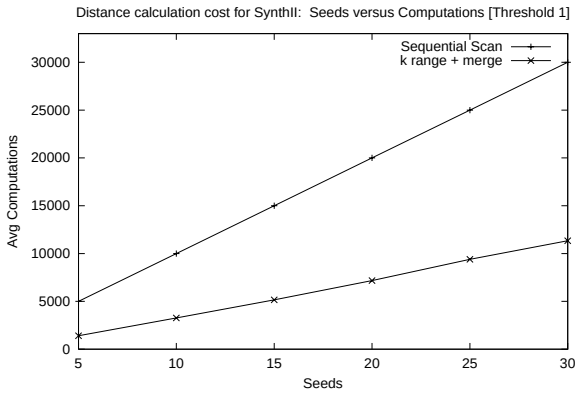
The two graphs in Figure 23 compare the elapsed wall-clock time and the pairwise distance computation costs incurred in searching the TWO_CIRCLES training data set for points within a variable aggregate dissimilarity threshold from the seeds. These results demonstrate that the k range queries can be efficiently merged and filtered.

The cross-over point where sequential scan performs as quickly as merging occurred at a threshold of approximately 0.9. Figure 26 shows that this threshold accepts approximately 30% of the database. It is our belief that users of large interactive databases will rarely be interested in queries of such low selectivity, and thus the merging method is worthwhile.

As shown in Figure 24, both measurements of cost appear to scale with the number of seeds. These results are taken without caching the distance computations from previous searches; these searches were all independent. These two graphs compare the elapsed wall-clock time and the pairwise distance computation costs incurred in searching the TWO_CIRCLES training



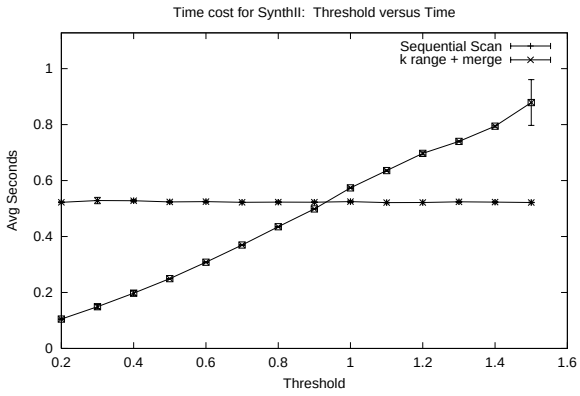
Elapsed time



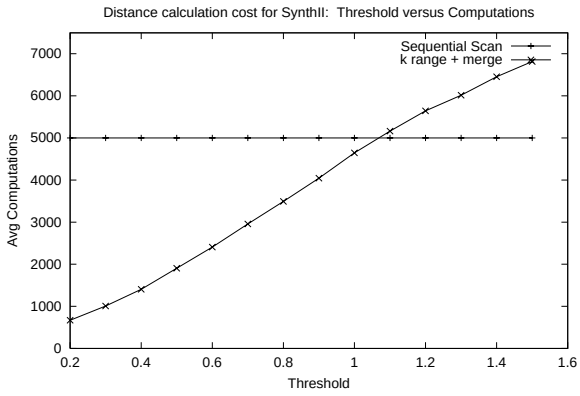
Distance Computations

Figure 24: Time and computational cost for TWO_CIRCLES, varying seeds.

data set for points with aggregate dissimilarity less than or equal to 1, varying the number of seeds.



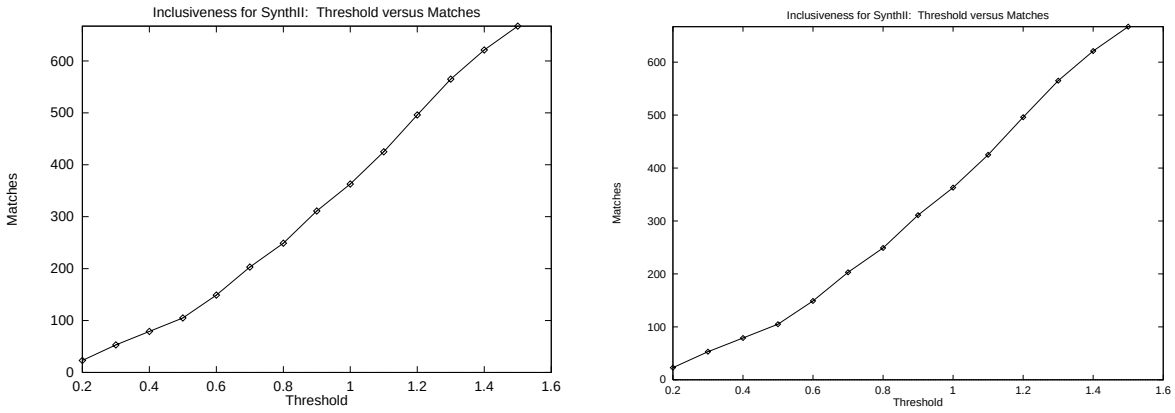
Elapsed time



Distance Computations

Figure 25: Time and computational cost on TWO_CIRCLES, with L_∞.

Figure 25 shows results for the same test, but with the underlying pairwise distance function L_∞, not L_2. The trends are quite similar.



L_2 L_∞

Figure 26: Threshold versus Selectivity, with $d = L_2$ and $d = L_\infty$.

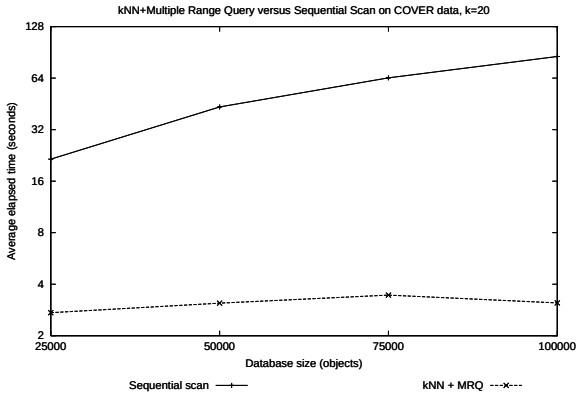
6.7 k-NN Queries

Likewise, our k-NN algorithm, built upon on our range query algorithm, is far faster than sequential scanning – and this advantage increases with the size of the database. This is as expected. Sequential scanning, naturally, scales linearly, while our algorithm is more concerned with k and $|G|$, as these determine how many queries we must execute. M-trees and Slim-trees are dynamic metric indices, so we are not concerned with the one-time cost of building the structure – a full rebuild should be quite rare compared to the number and cost of queries (Ciaccia *et al.*, 1997; Traina Jr. *et al.*, 2000).

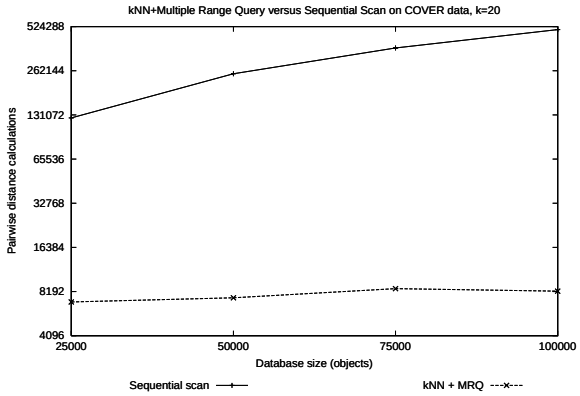
Figures 27 and 28 show elapsed wall-clock time and computational costs for 20- and 50-NN as a function of database size. Sequential scanning naturally does not scale; our algorithm, on the other hand, scales very well.

7 Conclusions

We proposed a feedback-driven method to handle queries expressed in terms of multiple positive examples, on arbitrary vector or even metric databases.

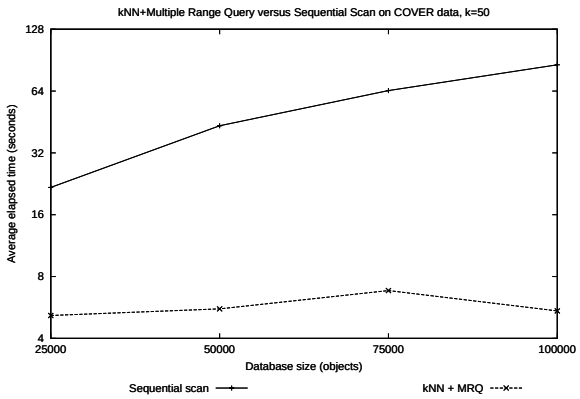


Elapsed time

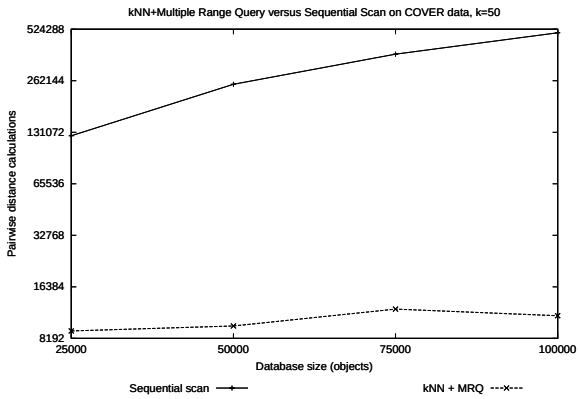


Distance Computations

Figure 27: Time and computational cost for 20-NN on the COVER data. Note that the y-axes are *logarithmic*. Increasing the database size has the expected effects on sequential scanning – it scales linearly, after all – while the nearest-neighbor algorithm is relatively unaffected. The time results are averages, in order to compensate partly for the inequities of a multitasking scheduler; the distance calculates are also averages, but as the algorithms used are deterministic they did not vary.



Elapsed time



Distance Computations

Figure 28: Time and computational cost for 50-NN on the COVER data. Again, the y-axes are *logarithmic*. We note that the results are very similar to those for 20-NN.

Our method applies to general metric spaces, as the distance combination function depends on only the pairwise distances and not the actual nature of the data or the pairwise distance function. In addition, it handles disjunctive and concave queries that are fundamentally *impossible* for traditional relevance feedback methods. A user can, without any domain-specific query language, specify a disjunctive query merely by labelling as "good" objects representative of the different classes. We argue that this combination of broad applicability, power and ease-of-use makes this method more valuable than other systems existing today.

The heart of our method is the FALCON aggregate dissimilarity, or $D_{\mathcal{G}}$, which is able to "learn" disjunctive queries via relevance feedback. Additional contributions include the following:

- Theorem 1. We use this to build algorithms that, with an index supporting single-query-point range and nearest-neighbor queries, scalably perform multi-query-point versions of the same.

- Experiments that show that the aforementioned algorithms scale very well with regard to database size.

- Experiments on real and synthetic data, that show that our proposed method, FALCON, achieves good precision and recall. For instance, with all queries, $\alpha = -5$ yielded at least 80% precision at 50% recall with 10 iterations.

- Experiments that show that FALCON needs at most 10 feedback iterations to reach high precision/recall, and will reach a "steady state" for all levels of recall below 100% within approximately 20 iterations. Beyond 20 iterations, minor progress is made for 100% recall.

- Experiments that show that a good range for α includes -10 to -2; on many queries, the sensitivity of the performance on α is low.

Possible avenues for future work include using $D_{\mathcal{G}}$ for other data mining tasks, such as classification and representative selection (M.K. Markey & Murphy, 1999).

Acknowledgements

This research was partially funded by the National Science Foundation under Grants No. DMS-9873442, and IIS-9910606; also, by DARPA/ITO through Order F463, issued by ESC/ENS under contract N66001-97-C-851; and the Office of Naval Research Grant N-00014-96-1-1222. Additional funding was provided by donations from NEC and Intel. Views and conclusions contained in this document are those of the authors and should not be interpreted as representing official policies, either expressed or implied, of the Defense Advanced Research Projects Agency or of the United States Government.

All trademarks are property of their respective owners.

A Aggregate Dissimilarity and Fuzzy Boolean Expressions

Let $d_i = d(x, g_i)$, for $i = 1..n$, be the n pairwise distances of object x from each object g_i in the selected set $\mathcal{G}$. Our justification of Equation 1 starts with considering similarities rather than distances.

$$s_i = s(x, g_i),\ i = 1..n \tag{8}$$

be the pairwise similarity values for the object x and each object in $\mathcal{G}$. Also, let $S_{\mathcal{G}}(x)$ denote the similarity (or "desirability") of object x with respect to the selected objects in $\mathcal{G}$. Let these aggregate similarities take positive values, with higher values meaning higher desirability.

We can link similarities with distances with any monotonically decreasing function; see (Ishikawa *et al.*, 1998) for some options. Using the simplest option

$$s_i = \frac{1}{d_i} \tag{9}$$

and

$$S_{\mathcal{G}}(x) = \frac{1}{D_{\mathcal{G}}(x)} \tag{10}$$

we see that Equation 1 is, implicitly, combining similarities in a fashion related to a fuzzy "OR" as discussed in (Salton *et al.*, 1983).

B Aggregate Dissimilarity when $\alpha = 2$

In this section, we prove that $D_{\mathcal{G}}$ is equivalent to a standard centroid-based single-query-point method when $\alpha = 2$.

Let $\vec{x} = (x_1, ..., x_d)$ be a d-dimensional candidate to be scored with respect to $\mathcal{G} = \{\vec{g_i}\}$, with $|\mathcal{G}| = n$ and each $\vec{g_i}$ is a d-dimensional vector $\vec{g_i} = (g_{i,1}, ..., g_{i,d})$.

Let $\vec{\mu} = (\mu_j) = \left(\frac{1}{n}\sum_i^n g_{i,j}\right)$ be the mean of the vectors in $\mathcal{G}$ where μ_j is the mean of all $g_{i,j}$ for $i \in [1, n]$. Also, let σ_j be the standard deviation of all $g_{i,j}$ for $i \in [1, n]$.

Then:

$$\begin{aligned}
\sum_{i=1}^{n} |(\vec{x} - \vec{g_i})|^2 &= \sum_{i=1}^{n}\sum_{j=1}^{d} (x_j - g_{i,j})^2 \\
&= \sum_{i=1}^{n}\sum_{j=1}^{d} \left(x_j^2 - 2x_j g_{i,j} + g_{i,j}^2\right) \\
&= \sum_{j=1}^{d} \left(n x_j^2 - 2x_j \sum_{i=1}^{n} g_{i,j} + \sum_{i=1}^{n} g_{i,j}^2\right) \\
&= \sum_{j=1}^{d} \left(n x_j^2 - 2n x_j \vec{\mu}_j + \sum_{i=1}^{n} g_{i,j}^2\right) \\
&= \left(n\,|\vec{x}|^2\right) - 2n\,(\vec{x} \cdot \vec{\mu}) + \sum_{i=1}^{n} |\vec{g_i}|^2 \\
&= n\left(|\vec{x}|^2 - 2\,(\vec{x} \cdot \vec{\mu})\right) + \sum_{i=1}^{n} |\vec{g_i}|^2 \\
&= n\,(|\vec{x}| - |\vec{\mu}|)^2 + \sum_{i=1}^{n} |\vec{g_i}|^2 - n\,|\vec{\mu}|^2 \\
\sum_{i=1}^{n} |(\vec{x} - \vec{g_i})|^2 &= n\,(|\vec{x}| - |\vec{\mu}|)^2 + n\left(\sum_{j=1}^{d} \sigma_j^2\right)
\end{aligned}$$

Therefore, when $\alpha = 2$, the only impact of the choice of x on the scoring function is to change the magnitude of the distance from x to the centroid. In other words, it becomes a single-query-point function that chooses the centroid as the ideal query point.

References

Beckmann, N., Kriegel, H.-P., Schneider, R., & Seeger, B. 1990. The R*-tree: An Efficient and Robust Access Method for Points and Rectangles. *ACM SIGMOD*, May 23-25, 322–331.

Beyer, Kevin, Goldstein, Jonathan, Ramkrishnan, Raghu, & Shaft, Uri. 1998. When is "Nearest Neighbor" Meaningful? ICDT.

Brinkhoff, Thomas, Kriegel, Hans-Peter, & Seeger, Bernhard. 1993 (May 26-28). Efficient Processing of Spatial Joins Using R-trees. *Pages 237–246 of: Proc. of ACM SIGMOD.*

Buckley, Chris, Salton, Gerard, & Allan, James. 1994. The effect of adding relevance information in a relevance feedback environment. *Pages 292–300 of: Proc. of SIGIR.*

Chiueh, Tzi-cker. 1994. Content-Based Image Indexing. *Pages 582–593 of:* Bocca, Jorge B., Jarke, Matthias, & Zaniolo, Carlo (eds), *Proc. of 20th International Conference on Very Large Data Bases.* Morgan Kaufmann.

Christel, M., Kanade, T., Mauldin, M., Reddy, R., Sirbu, M., Stevens, S., & Wactlar, H. 1995. Informedia Digital Video Library. *Communication of the ACM*, **38**(4), 57–58.

Ciaccia, Paolo, Patella, Marco, & Zezula, Pavel. 1997. M-tree: An Efficient Access Method for Similarity Search in Metric Spaces. *VLDB*, 426–435.

Cox, Ingemar J., Miller, Matt L., Omohundro, Stephen M., & Yianilos, Peter M. 1996. PicHunter: Bayesian Relevance Feedback for Image Retrieval. *In: Proceedings of International Conference on Pattern Recognition.*

Cox, Ingemar J., Miller, Matthew L., Minka, Thomas P., & Yianilos, Peter N. 1998. An optimized interaction strategy for Bayesian relevance feedback. *Pages 553–558 of: Proc. of IEEE Conf. on Comp. Vis. and Pattern Recognition.*

Faloutsos, C., Barber, R., Flickner, M., Hafner, J., Niblack, W., Petkovic, D., & Equitz, W. 1994. Efficient and Effective Querying by Image Content. *Journal of Intelligent Information Systems*, **3**(3/4), 231–262.

Gaede, Volker, & Günther, Oliver. 1998. Multidimensional access methods. *ACM Computing Surveys*, **30**(2), 170–231.

Guttman, A. 1984 (June). R-Trees: A Dynamic Index Structure for Spatial Searching. *Pages 47–57 of: Proc. ACM SIGMOD.*

Ishikawa, Yoshiharu, Subramanya, Ravishankar, & Faloutsos, Christos. 1998. *MindReader: Querying databases through multiple examples.* Tech. rept.

Jagadish, H. V., Koudas, Nick, & Muthukrishnan, S. 1999. Mining Deviants in a Time Series Database. *Pages 102–113 of:* Atkinson, Malcolm P., Orlowska, Maria E., Valduriez, Patrick, Zdonik, Stanley B., & Brodie, Michael L. (eds), *Proc. of 25th International Conference on Very Large Data Bases.* Morgan Kaufmann.

Katayama, Norio, & Satoh, Shin'ichi. 2000. Similarity image retrieval with significance-sensitive nearest-neighbor search. *Pages 177–186 of: Proc. of Sixth International Workshop on Multimedia Information Systems (MIS2000).*

Keogh, Eamonn J., & Pazzani, Michael J. 1998. An enhanced representation of time series which allows fast and accurate classification, clustering and relevance feedback. *In: AAAI Workshop on Predicting the Future: AI Approaches to Time-Series Analysis.*

Keogh, Eamonn J., & Pazzani, Michael J. 1999. Relevance Feedback Retrieval of Time Series Data. *Pages 183–190 of: Proceedings of the Annual ACM Conference on Research and Development in Information Retrieval.* Berkeley, CA: ACM.

M.K. Markey, M.V. Boland, & Murphy, R.F. 1999. Towards Objective Selection of Representative Microscope Images. *Biophys. J.* in press.

Murphy, P.M., & Aha, D.W. 1994. *UCI Repository of Machine Learning Databases*. Department of Information and Computer Science, University of California, Irvine, CA. [http://www.ics.uci.edu/∼mlearn/MLRepository.html].

Pagel, Bernd-Uwe, Korn, Flip, & Faloutsos, Christos. 2000. Deflating the Dimensionality Curse using Multiple Fractal Dimensions. *Pages 589–598 of: Proceedings of the 16th International Conference on Data Engineering*. San Diego, CA: IEEE Computer Society.

Prokaew, Kriengkrai, Mehrotra, Sharad, Ortega, Michael, & Chakrabarti, Kaushik. 1999 (June). Similarity Search Using Multiple Examples in MARS. *In: 1999 International Conference on Visual Information Systems*.

Rocchio, Joseph John. 1971. Relevance Feedback in Information Retrieval. *Pages 313–323 of:* Salton, Gerard (ed), *The SMART Retrieval System – Experiments in Automatic Document Processing*. Englewood Cliffs, N.J.: Prentice Hall.

Rui, Y., Huang, T., Ortega, M., & Mehrotra, S. 1998a. Relevance feedback: A power tool in interactive content-based image retrieval. *IEEE Transactions on Circuits and Systems for Video Technology*, **8**(5), 644–655.

Rui, Yong, Huang, Thomas S., & Mehrotra, Sharad. 1997 (Oct.). Content-based Image Retrieval with Relevance Feedback in MARS. *In: Proceedings of IEEE International Conference on Image Processing '97*.

Rui, Yong, Huang, Thomas S., & Mehrotra, Sharad. 1998b (Jan.). Human Perception Subjectivity and Relevance Feedback in Multimedia Information Retrieval. *In: Proceedings of IS&T and SPIE Storage and Retrieval of Image and Video Databases VI*.

Salton, G., Fox, E.A., & Wu, H. 1983. Extended Boolean Information Retrieval. *CACM*, **26**(11), 1022–1036.

Traina Jr., Caetano, Traina, Agma J. M., Seeger, Bernhard, & Faloutsos, Christos. 2000. Slim-Trees: High Performance Metric Trees Minimizing Overlap Between Nodes. *Pages 51–65 of:* Zaniolo, Carlo, Lockemann, Peter C., Scholl, Marc H., & Grust, Torsten (eds), *Proc. of 7th International Conference on Extending Database Technology.* Lecture Notes in Computer Science, vol. 1777. Springer.

Wu, Leejay, Faloutsos, Christos, Sycara, Katia, & Payne, Terry R. 2000. *FALCON: Feedback Adaptive Loop for Content-Based Retrieval.* Tech. rept. Carnegie Mellon University.

Zhang, Bin. 2001. Generalized K-Harmonic Means – Dynamic Weighting of Data in Unsupervised Learning. *In: Proc. of the First SIAM International Conference on Data Mining.*